

CprE 3810: Computer Organization and Assembly-Level Programming

Project Part 2 Report

Team Members: Jonathan Clark

Project Teams Group #C02

Refer to the highlighted language in the project 1 instruction for the context of the following questions.

[1.a] Come up with a global list of the datapath values and control signals that are required during each pipeline stage.

Register vectors

IF/ID:

PC

PC Incremented

Instruction

ID/EX: (now we need all our control signals)

PC

PC Incremented

Instruction

RS1 Data

RS2 Data

Immediate

Control:

ALUCTL vector

jalr

jump

branch

auipc

alusrc

memWrite

byteOp

regWrite

WRBCKSEL

haltFlag

EX/MEM:

Instruction

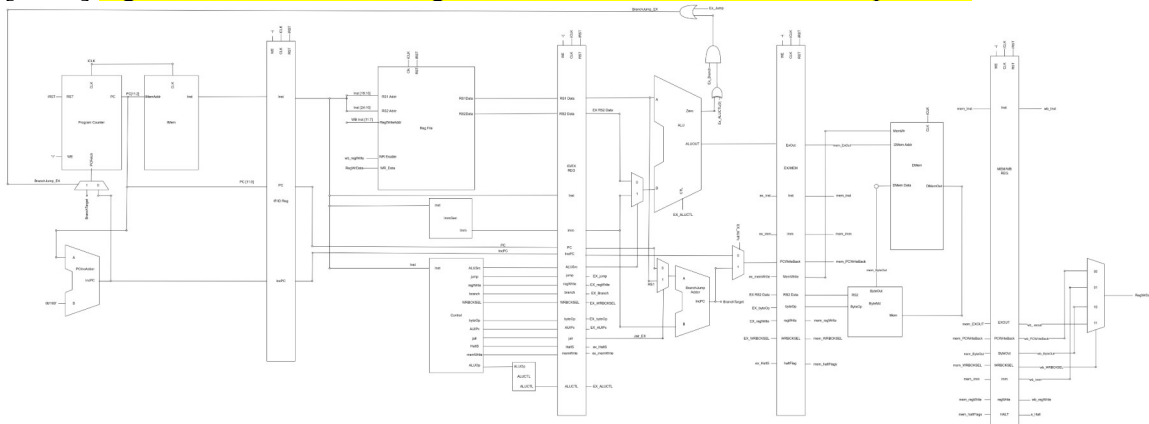
PCWriteBack (for jumps and auipc)
 ALUOut (all execute output)
 RS2Data (for stores)

control:
 memWrite
 byteOp
 regWrite
 WRBCKSEL
 haltFlag

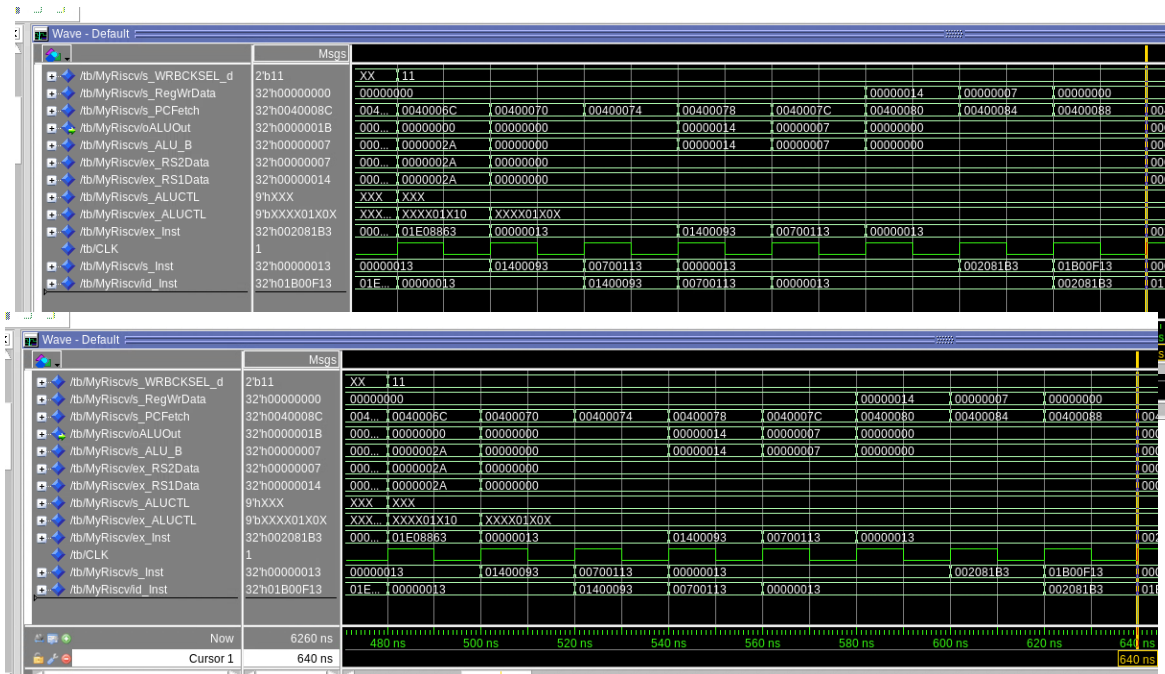
MEM/WB:
 Instruction
 PCWriteBack
 ALUOut
 ByteOut
 Immediate

control:
 regWrite
 WRBCKSEL
 haltFlag

[1.b.ii] high-level schematic drawing of the interconnection between components.



[1.c.i] include an annotated waveform in your writeup and provide a short discussion of result correctness.



```

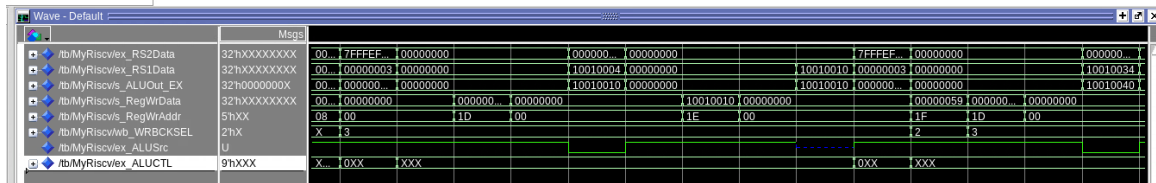
49
50 # ADDI
51     addi    x1, x0, 42
52     addi    x30, x0, 42
53     nop
54     nop
55     nop
56     beq     x1, x30, addi_pass
57     nop
58     nop
59     wfi
60 addi_pass:
61
62 # ADD
63     addi    x1, x0, 20
64     addi    x2, x0, 7
65     nop
66     nop
67     nop
68     add     x3, x1, x2

```

Our nops have a few goals with this program, the first 3 are to protect the beq from the data hazard created from writing to x30 and comparing it to x1. X1 also has a data hazard but we don't need to account for it because its handled by the nops for the x30 data hazard. We see these addis, at 340 and 360 ns, and then the nops from 380-440 ns. In this time we are waiting for 42 or 2A to reach regwrdata which means its in write back. After this we can let the branch instruction into decode to pick up the accurate value. We see this at 460 ns, when 01E08863 (the beq inst) enters decode on the id_inst signal. By this time we can see 2A has already been in regwrdata for both the x1 write cycle and the x30 write cycle and it has completed write back. We then see 2 nops to wait for the pc to pick up the updated value provided by the branch. We then see a similar situation to before where we add 20 to x1 and then 7 to x2, then we use those values in the next add. We

need to wait for 7 to reach and leave regwdata before the add instruction 002081B3 can enter decode on id_inst. And that's what we see in the second screenshot, 14 at 580ns in regwdata, followed by 7 at 600 ns, and then the instruction 002081B3 enters decode and will pick up those values

[1.c.ii] Include an annotated waveform in your writeup of two iterations or recursions of these programs executing correctly and provide a short discussion of result correctness. In your waveform and annotation, provide 3 different examples (at least one data-flow and one control-flow) of where you did not have to use the maximum number of NOPs.



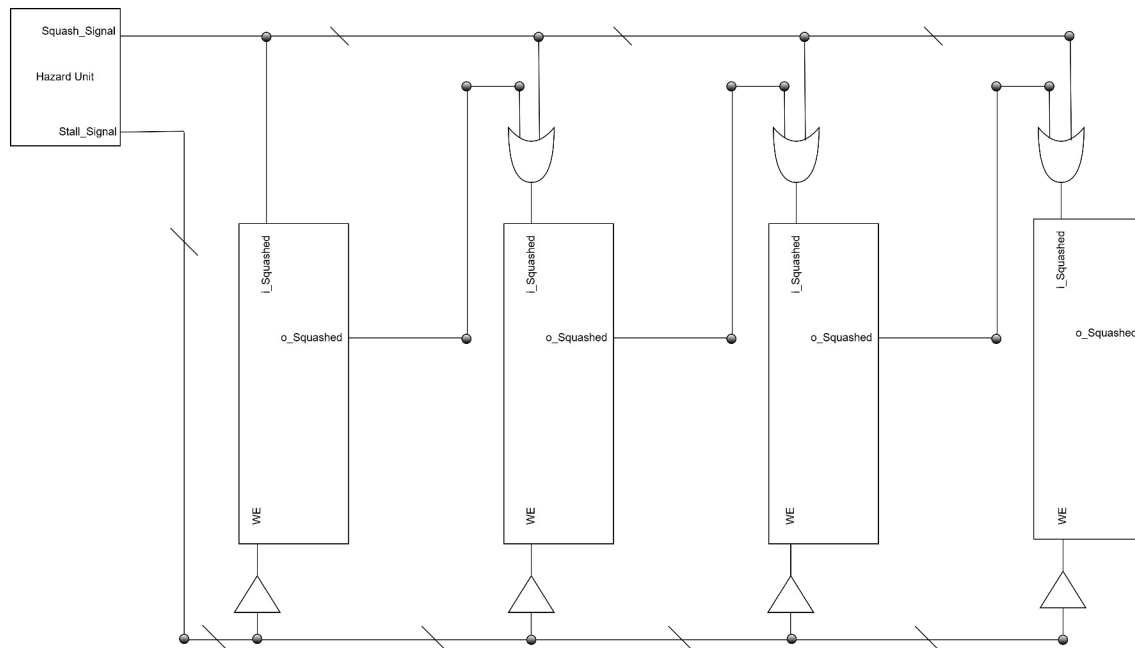
[1.d] report the maximum frequency your software-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

47.78mhz

My critical path is a taken branch equals,

It starts from the id/ex reg, rs2 data passes through the source mux, then into the ALU, Moves through the adder, then the zero bit or gate at the end to determine if the subtraction is 0, then through the logic on the zero bit to determine a branch being taken, then that signal reaches the mux that feeds the pc the new value so that it can take in the branch value.

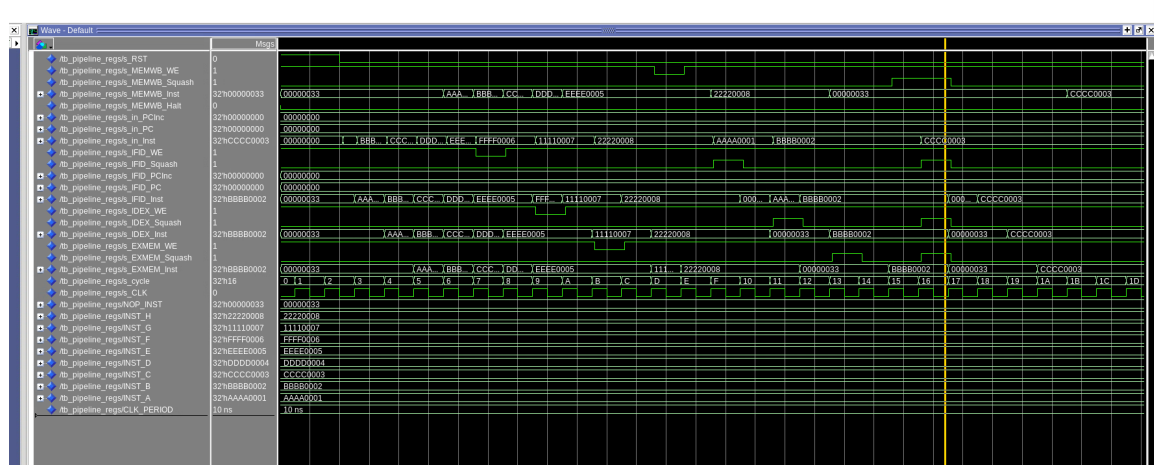
[2.a.ii] Draw a simple schematic showing how you could implement stalling and flushing operations given an ideal N-bit register.



The idea is that we seed a squash using the squash bit, and then if that registers squash bit is set high, then it will instead of taking in the values from the previous stage take in the values that will coincide with a nop. It will also importantly set an internal squash bit to

true, which will be output to the next regs squash input. This way we can seed a squash and it will propagate. For stalls we can just disable write.

[2.a.iii] Create a testbench that instantiates all four of the registers in a single design. Show that values that are stored in the initial IF/ID register are available as expected four cycles later, and that new values can be inserted into the pipeline every single cycle. Most importantly, this testbench should also test that each pipeline register can be individually stalled or flushed.



[2.b.i] list which instructions produce values, and what signals (i.e., bus names) in the pipeline these correspond to.

Technically all signals are produced and consumed so I wont go through all but the relevant ones are rs1 and rs2 data as well as regwritedata. If an instruction is writing to a register and we read it before write back is completed we need to forward that.

The difference is when they resolve this regwritedata, I'm consolidating lui, auipc, and alu operations to an execute output signal exOut so our produced signals are ex_exOut, mem_exOut, and mem_byteOut,

ex_exOut and mem_exOut producers are the same just in different positions:

add, addi, and, andi, lui, xor, xori, or, ori, slt, slti, sltiu, sll, srl, sra, sub, slli, srli, srai, auipc

mem_byteOut:

lw, lb, lh, lbu, lhu

We also technically have our pcwriteback signal which goes to regwritedata but the only thing we have to worry about there is jumps and on jumps we squash two cycles which is enough for it to reach write back

[2.b.ii] List which of these same instructions consume values, and what signals in the pipeline these correspond to.

All instructions besides wfi consume a register value. We will just forward to i_RS1Data, and i_RS2Data on the ID_EX reg

[2.b.iii] generalized list of potential data dependencies. From this generalized list, select those dependencies that can be forwarded (write down the corresponding pipeline stages that will be forwarding and receiving the data), and those dependencies that will require hazard stalls.

We have the phases of the pipeline segregated properly so that modules will only be attempted to be used by one instruction per cycle so the main issue is RAW. The entire processor is a state machine so all we need to do is make sure that the state for each

instruction is accurate to what the programmer expects from software. The only way this state will actually change is through the registers and the memory. A memory raw seems at first like a potential hazard but because of the riscv philosophy around memory load and stores, the closest we can come to a data hazard is a load immediately after a store. Since we both write and read memory in the mem stage, its not possible for this to get out of order. So we only really have to worry about register data dependencies. For this we will look at instructions that I mentioned above that either resolve their value for RD in execute or in memory as this will change how we forward. If they resolve in execute then we can easily forward whether they are the next cycle or theres a one cycle gap (past that we handle it in write back). If they are a load type, they resolve in memory and we must stall

[2.b.iv] global list of the datapath values and control signals that are required during each pipeline stage

same as software but now with the squash signal

[2.c.i] list all instructions that may result in a non-sequential PC update and in which pipeline stage that update occurs.

Beq, bne, blt, bge, bltu, bgeu, jal, jalr. All of these will resolve in execute and updating the pc with them was already handled in the software scheduled pipeline but now we need to squash since we are not adding nops

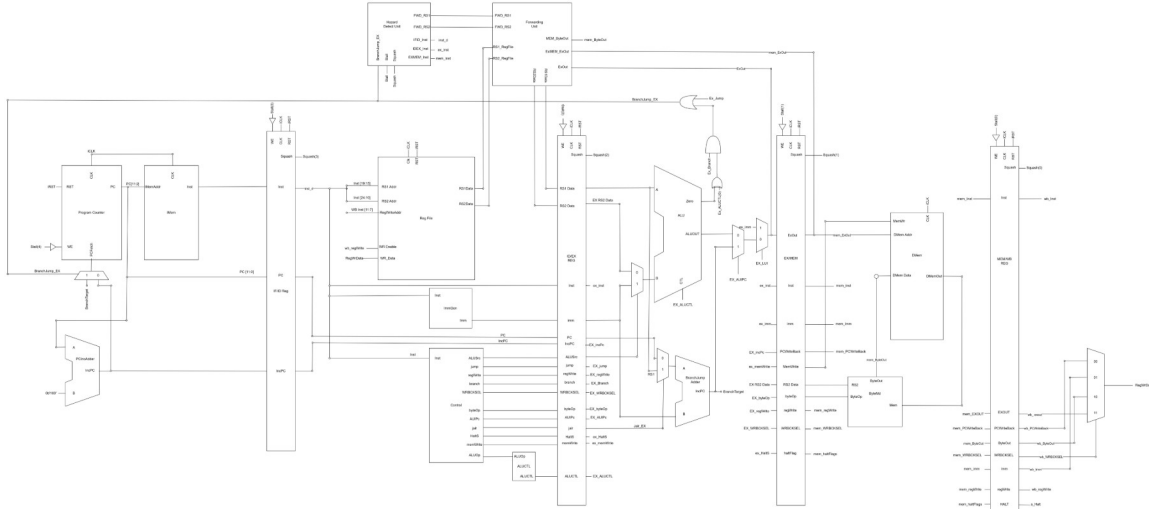
[2.c.ii] For these instructions, list which stages need to be stalled and which stages need to be squashed/flushed relative to the stage each of these instructions is in.

Nothing needs to be stalled, we could technically stall the pc after reading in one of these instructions and just let it pass up to execute then resume once the new pc is found but thats more complicated as well as would cut performance. Given a branch was not taken we would have wasted multiple cycles.

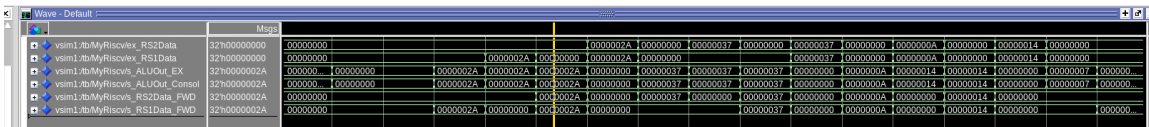
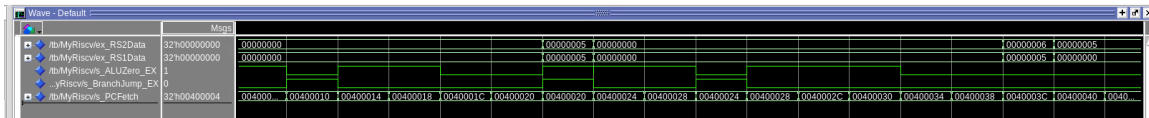
The simplest approach is just to always allow all these instructions to flow through, and on a branch or jump taken we will squash the IF/ID and ID/EX registers. We can let the instruction flow up through this point because like I said above the cpu is just a state machine and until they reach write back or a forwarded signal they will not affect that state nonlocally. So they will only impact the values of their sequential registers and nothing else. By letting them through to decode and squashing them it has the same affect

as stalling and inserting nops while the branch moves up to execute except this way we can keep them if the branch isn't taken.

[2.d] implement the hardware-scheduled pipeline using only structural VHDL. As with the previous processors that you have implemented, start with a high-level schematic drawing of the interconnection between components.



[2.e – i, ii, and iii] In your writeup, show the QuestaSim output for each of the following tests, and provide a discussion of result correctness. It may be helpful to also annotate the waveforms directly.



We can see the signal being forwarded from the alu to the appropriate rs1/rs2 signal

[2.e.i] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

Test #	Hazard Type	Forward Source	Forward Destination	Verified Behavior
1	EX RAW	EX result	ID/EX.rs1_val	Immediate dependency on RS1 resolved without stall
2	EX RAW	EX result	ID/EX.rs2_val	Immediate dependency on RS2 resolved without stall
3	Dual-source EX RAW	EX result	ID/EX.rs1_val + ID/EX.rs2_val	Same forwarded value feeds both operands
4	EX/MEM RAW	EX/MEM result	ID/EX.rs1_val	One-cycle delayed dependency resolved
5	EX/MEM RAW	EX/MEM	ID/EX.rs2_val	One-cycle delayed dependency

Test #	Hazard Type	Forward Source	Forward Destination	Verified Behavior
		result		resolved
6	Load-use RAW	MEM load result	ID/EX.rs1_val after stall	Hazard detector inserts 1-cycle stall correctly
7	Load-use RAW	MEM load result	ID/EX.rs2_val after stall	Load-use handling works on RS2
8	Delayed load RAW	MEM load result	ID/EX.rs1_val	Forwarding works without stall when timing allows
9	Chained RAWs	EX result	ID/EX.rs1_val repeatedly	Cascaded forwarding remains correct
10	Mixed simultaneous RAW	EX + EX/MEM	ID/EX.rs1_val + ID/EX.rs2_val	Independent mux selections work simultaneously
11	Mixed load stall + forward	MEM + EX/MEM	ID/EX.rs1_val + ID/EX.rs2_val	Stall and forwarding cooperate correctly
12	AUIPC RAW	AUIPC EX result	ID/EX.rs1_val	Non-ALU producer participates in forwarding
13	x0 exclusion	EX result	none	x0 excluded from forwarding logic

[2.e.ii] Create a spreadsheet to track these cases and justify the coverage of your testing approach. Include this spreadsheet in your report as a table.

BEQ	Taken	fail inst next slot	Passed, skipped inst
BEQ	Not Taken	Inst Next 2 slots	Passed, next inst executed
BNE	Taken	Fail inst next slot	Passed, skipped inst
BNE	Not Taken	Inst Next 2 slots	Passed, next inst executed
BGE	Taken	Fail inst next slot	Passed, skipped inst
BGE	Not Taken	Inst Next 2 slots	Passed, next inst executed
BLTU	Taken	Fail inst next slot	Passed, skipped inst
BGEU	Taken	Fail inst next slot	Passed, skipped inst
JAL	Taken	Jump backwards	Passed, jumped to label
JALR	Taken	Add immediately after	Passed, jumped to updated x0

[2.f] report the maximum frequency your hardware-scheduled pipelined processor can run at and determine what your critical path is (specify each module/entity/component that this path goes through).

Fmax was 48.85mhz for hardware, my critical path was a branch equals taken, which uses the adder/subtractor which is pretty slow
It started in my ID/EX reg, through the alusrc mux, through the adder/subtractor, then alu output select mux, into the zero bit or gate, and through the branch logic, back to the pc input mux